

Signett – Founding Product Engineer, Developer Adoption: What to Learn

Playwright & Cypress (MUST)

Both are browser end-to-end testing tools — you write a script that drives a real browser (click, type, navigate) and assert on what appears, instead of unit-testing a function in isolation. You already write React/Next.js apps; this is testing them the way a real user would.

- **Playwright** (Microsoft): one API drives Chromium, Firefox & WebKit. Auto-waits for elements (no manual sleeps), has a built-in “codegen” recorder, and a trace viewer for debugging failed runs. Bridges directly from your DevTools/browser debugging experience (Clink AI, Sentry work) — same mental model, now scripted.
- **Cypress**: older, Chromium-first, runs inside the browser itself (vs. Playwright’s out-of-process driver) — great DX, time-travel debugger, but no true cross-browser parallelism like Playwright.
- **The 20% that matters**: locators (`getByRole`, `getByText` — prefer accessible queries over CSS selectors), `await page.click()` / `fill()` / `expect()` assertions, and how both frameworks auto-retry assertions until they pass or time out. Skip advanced parallel-sharding/CI config for now.
- **Bridge**: you already reason about async UI state (Redux, Convex real-time sync) — Playwright/Cypress assertions are the same “wait for this to become true” pattern, just at the DOM level.

MCP – Model Context Protocol (MUST)

An open standard (Anthropic-originated, now broadly adopted) that lets an LLM call external tools/data sources through a common interface, instead of every app inventing its own bespoke function-calling schema. You already build the underlying pattern manually (Echo’s 3 Zod-typed tools, Vibe’s tool-calling loop) — MCP formalizes that into a reusable, transport-agnostic protocol.

- An MCP **server** exposes tools/resources/prompts; an MCP **client** (e.g. Claude Desktop, an IDE, or your own agent) discovers and calls them over a standard JSON-RPC-based transport (stdio locally, or HTTP/SSE remotely).
- The 20%: how a tool is declared (name, description, JSON-schema input) — this is almost identical to the Zod tool schemas you already write for LLM function calling, just expressed as MCP’s schema format.
- You already use Claude Code daily, which is an MCP client under the hood — read one existing MCP server’s source (e.g. a simple filesystem or GitHub MCP server) to see the shape end-to-end.

WebMCP (GOOD to know)

A newer, less-established idea: bringing MCP-style tool-calling directly into the **browser** so a webpage itself can expose “tools” an AI agent can discover and call while browsing it (as opposed to MCP servers living outside the browser). This is squarely what the JD’s “agent-ready websites” language is about.

- There is no single dominant spec yet — expect to read whatever the company’s own docs/RFC say rather than a settled standard. Treat it as: “MCP’s tool-calling contract, but declared by a webpage for an in-browser or extension-based agent to consume.”
- Bridge: your browser-API experience (DOM manipulation, live preview URLs in Vibe, real-time sync in Echo) is exactly the surface WebMCP tools would sit on top of.

Full-Stack & Product-Adoption Mindset (context, not a technology)

The JD is unusually explicit that this is NOT a “ship code from a backlog” role — it’s finding external developer teams, integrating into their real codebases, finding friction, and publishing what you learned. Nothing to “learn” technically here, but worth internalizing before an interview: be ready to talk about a time you dove into an unfamiliar codebase quickly (you already do this across your project portfolio)

and about writing clearly for other developers (documentation, not just code).